

Chapter 3 — Building and Preparing Datasets for Energetic Materials

Python Code Examples · Jupyter Notebook Edition

Finding, cleaning, and organising your data

Block	Topic
0	Install & shared dataset
1	Build a curated dataset from literature values
2	Structure standardisation — canonicalise and deduplicate
3	Explore the dataset — distributions and correlations
4	Detect and handle missing values
5	Detect and handle outliers
6	Descriptor calculation and feature matrix assembly
7	Feature preprocessing — variance filter, correlation filter, scaling
8	Train / test splitting — random and scaffold-based
9	Full pipeline end-to-end — from SMILES to ML-ready matrix

Run each cell in order — Shift+Enter.

Block 0 · Install & Shared Dataset

Run once. Creates the shared literature dataset used in all later blocks.

```
In [1]: # Uncomment to install on first run
# !pip install rdkit mordred pandas matplotlib seaborn scikit-learn
import warnings
warnings.filterwarnings('ignore')
import re, math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from rdkit import Chem, DataStructs
from rdkit.Chem import Descriptors, rdMolDescriptors, AllChem, MolToSmiles, MolFromSmiles
from rdkit.Chem.Scaffolds import MurckoScaffold
from rdkit.Chem import rdFingerprintGenerator

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.feature_selection import VarianceThreshold
from sklearn.impute import SimpleImputer

from IPython.display import display, HTML
```

```

%matplotlib inline
plt.rcParams['figure.dpi'] = 120

# — Literature dataset: 30 energetic compounds —————
# Columns: name, SMILES, formula, MW, density(g/cc),
#          impact_sens(J), decomp_temp(C), det_velocity(m/s),
#          heat_formation(kJ/mol), class
# Sources: Cooper 1996, Dobratz 1981, Zeman 2003, open literature

RAW_DATA = [
    # name,          SMILES,
    ['TNT',          'Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]',
    ['RDX',          'O=[N+]( [O- ] )N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1',
    ['HMX',          'O=[N+]( [O- ] )N1CN([N+](=O)[O-])CN([N+](=O)[O-])CN([N+](=O)[O-])C1',
    ['PETN',         'C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]',
    ['TATB',         'Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])c(N)c1[N+](=O)[O-]',
    ['NTO',          'O=[N+]( [O- ] )c1n[nH]c(=O)[nH]1',
    ['FOX-7',        'NC(=C([N+](=O)[O-])[N+](=O)[O-])N',
    ['DNAN',         'COc1ccc([N+](=O)[O-])cc1[N+](=O)[O-]',
    ['CL-20',        'O=[N+]( [O- ] )N1CN2CN([N+](=O)[O-])CN1[N+](=O)[O-]',
    ['Tetryl',       'CN([N+](=O)[O-])c1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]',
    ['TNAZ',         'O=[N+]( [O- ] )C1(CN([N+](=O)[O-])C1[N+](=O)[O-])[N+](=O)[O-]',
    ['LLM-105',      'Nc1nc2c([N+](=O)[O-])n[nH]c2c([N+](=O)[O-])n1',
    ['ANTA',         'Nc1nnc([N+](=O)[O-])[nH]1',
    ['Nitroguanidine', 'NC(=N)N[N+](=O)[O-]',
    ['BTNEU',        'O=[N+]( [O- ] )c1c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])',
    ['Picric acid',  'Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]',
    ['DATB',         'Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])cc1[N+](=O)[O-]',
    ['NONA',         'O=[N+]( [O- ] )c1c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])',
    ['BTF',          'O=[N+]( [O- ] )c1c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])c([N+](=O)[O-])',
    ['AN',           'N.[N+](=O)([O-])[O-]',
    ['AP',           '[NH4+].[O-]Cl(=O)=O',
    ['NC13',         'OCC1OC(OC2C(COC(=O)[N+](=O)[O-])OC(OC3C(COC(=O)[N+](=O)[O-])OC(OC4C(COC(=O)[N+](=O)[O-])OC4)OC3)OC2)OC1',
    ['NG',           'O[C@@H](CO[N+](=O)[O-])CO[N+](=O)[O-]',
    ['EDNA',         'O=[N+]( [O- ] )NCC[N+](=O)[O-]',
    ['Heptyl',       'CC(C)(C)c1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]',
    ['TETNB',        'O=[N+]( [O- ] )c1c([N+](=O)[O-])c([N+](=O)[O-])cc([N+](=O)[O-])c1',
    ['MHN',          'OC(CO[N+](=O)[O-])CO[N+](=O)[O-]',
    ['NQ',           'NC(=N)N[N+](=O)[O-]',
    ['HNS',          'O=[N+]( [O- ] )c1ccc(C=Cc2ccc([N+](=O)[O-])cc2[N+](=O)[O-])c([N+](=O)[O-])',
    ['DNAZ',         'O=[N+]( [O- ] )N1CC([N+](=O)[O-])C1',
]

COLS = ['Name', 'SMILES', 'Formula', 'MW', 'Density', 'ImpactSens_J',
        'DecompTemp_C', 'DetVel_ms', 'HeatForm_kJmol', 'Class']

df_raw = pd.DataFrame(RAW_DATA, columns=COLS)

print(f'Dataset loaded: {df_raw.shape[0]} compounds x {df_raw.shape[1]} columns')
print(f'Classes: {df_raw["Class"].value_counts().to_dict()}')
"""
display(df_raw.head(6).style.hide(axis='index')
        .set_caption('Raw Literature Dataset (first 6 rows)')
        .set_table_styles([{'selector': 'caption',
                             'props': [('font-size', '14px'), ('font-weight', 'bold')]}])
"""

display(df_raw)

Dataset loaded: 30 compounds x 10 columns
Classes: {'Secondary': 18, 'Insensitive': 9, 'Propellant': 3}

```

	Name	SMILES	Formula	MW	Density	ImpactSens_J	Decomposition
0	TNT	<chem>Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]</chem>	C7H5N3O6	227.13	1.654	15.0	
1	RDX	<chem>O=[N+](O-)[N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1]</chem>	C3H6N6O6	222.12	1.820	7.4	
2	HMX	<chem>O=[N+](O-)[N1CN([N+](=O)[O-])CN([N+](=O)[O-])...]</chem>	C4H8N8O8	296.16	1.905	7.4	
3	PETN	<chem>C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])CO[N+](=O)[O-]</chem>	C5H8N4O12	316.14	1.778	3.0	
4	TATB	<chem>Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])c(N)c1[N+](=O)[O-]</chem>	C6H6N6O6	258.15	1.939	50.0	
5	NTO	<chem>O=[N+](O-)[c1n[nH]c(=O)[nH]1]</chem>	C2H2N4O3	130.06	1.930	71.0	
6	FOX-7	<chem>NC(=C([N+](=O)[O-])[N+](=O)[O-])N</chem>	C2H4N4O4	148.08	1.885	25.0	
7	DNAN	<chem>COc1ccc([N+](=O)[O-])cc1[N+](=O)[O-]</chem>	C7H6N2O5	198.13	1.321	40.0	
8	CL-20	<chem>O=[N+](O-)[N1CN2CN([N+](=O)[O-])CN1[N+](=O)[O-]2]</chem>	C6H6N12O12	438.19	2.044	4.0	
9	Tetryl	<chem>CN([N+](=O)[O-])c1c([N+](=O)[O-])cc([N+](=O)[O-])cc1</chem>	C7H5N5O8	287.14	1.730	3.0	
10	TNAZ	<chem>O=[N+](O-)[C1(CN([N+](=O)[O-])C1[N+](=O)[O-])...]</chem>	C3H4N4O8	240.08	1.840	12.0	
11	LLM-105	<chem>Nc1nc2c([N+](=O)[O-])n[nH]c2c([N+](=O)[O-])n1</chem>	C4H4N8O4	228.12	1.913	40.0	
12	ANTA	<chem>Nc1nnc([N+](=O)[O-])[nH]1</chem>	C2H3N5O2	129.08	1.840	60.0	
13	Nitroguanidine	<chem>NC(=N)N[N+](=O)[O-]</chem>	CH4N4O2	104.07	1.770	95.0	
14	BTNEU	<chem>O=[N+](O-)[c1c([N+](=O)[O-])c([N+](=O)[O-])c(...)]</chem>	C6N6O12	348.10	1.980	8.0	
15	Picric acid	<chem>Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]</chem>	C6H3N3O7	229.10	1.763	7.4	
16	DATB	<chem>Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])cc1[N+](=O)[O-]</chem>	C6H5N5O6	243.13	1.840	50.0	
17	NONA	<chem>O=[N+](O-)[c1c([N+](=O)[O-])c([N+](=O)[O-])c(...)]</chem>	C6HN5O10	287.10	1.870	6.0	
18	BTF	<chem>O=[N+](O-)[c1c([N+](=O)[O-])c([N+](=O)[O-])c(...)]</chem>	C5N6O12	334.10	1.920	5.0	
19	AN	<chem>N.[N+](=O)[O-]</chem>	H4N2O3	80.04	1.720	100.0	

	Name	SMILES	Formula	MW	Density	ImpactSens_J	Decomp
20	AP	[NH4+].[O-]Cl(=O)(=O)=O	ClH4NO4	117.49	1.950	110.0	
21	NC13	OCC1OC(OC2C(COC(=O) [N+](=O) [O-]))OC(OC3C(COC(=O...	C12H14N7O27	609.21	1.660	30.0	
22	NG	O[C@@H](CO[N+](=O) [O-])CO[N+](=O)[O-]	C3H5N3O9	227.09	1.591	0.2	
23	EDNA	O=[N+](([O-])NCC[N+] (=O)[O-])	C2H5N3O4	135.08	1.713	5.0	
24	Heptyl	CC(C)(C)c1c([N+](=O) [O-])cc([N+](=O) [O-])cc1[N+]	C10H11N3O6	269.21	1.490	20.0	
25	TETNB	O=[N+](([O-])c1c([N+](=O) [O-])c([N+](=O)[O-])cc...	C6H2N4O8	258.10	1.814	8.0	
26	MHN	OC(CO[N+](=O) [O-])CO[N+](=O)[O-]	C3H6N2O7	182.09	1.450	60.0	
27	NQ	NC(=N)N[N+](=O)[O-]	CH4N4O2	104.07	1.710	NaN	
28	HNS	O=[N+] ([O-])c1ccc(C=Cc2ccc([N+] (=O)[O-])cc2[N+]	C14H6N6O12	450.22	1.745	7.0	
29	DNAZ	O=[N+](([O-])N1CC([N+] (=O)[O-])C1	C3H5N3O4	147.09	1.699	14.0	

Block 1 · Literature Data Quality Audit

Before doing anything else, **audit your data**:

- How many records do you have per property?
- Which properties have the most missing values?
- Are units consistent?
- What is the class distribution?

A data audit tells you where to focus cleaning effort.

```
In [2]: # — 1a: Missing value audit —

numeric_cols = ['MW', 'Density', 'ImpactSens_J', 'DecompTemp_C',
                'DetVel_ms', 'HeatForm_kJmol']

# Convert to numeric (coerce bad values to NaN)
df = df_raw.copy()
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Missing value summary
missing = pd.DataFrame({
    'Total': df[numeric_cols].shape[0],
    'Missing': df[numeric_cols].isna().sum(),
    'Present': df[numeric_cols].notna().sum(),
    'Missing%': (df[numeric_cols].isna().mean() * 100).round(1),
})
```

```
}).reset_index().rename(columns={'index': 'Property'})

"""
display(missing.style
        .background_gradient(subset=['Missing%'], cmap='Reds')
        .hide(axis='index')
        .set_caption('Missing Value Audit')
        .set_table_styles([{'selector': 'caption',
                             'props': [('font-size', '14px'), ('font-weight', 'bold')]}])
"""
display(missing)
```

	Property	Total	Missing	Present	Missing%
0	MW	30	0	30	0.0
1	Density	30	0	30	0.0
2	ImpactSens_J	30	1	29	3.3
3	DecompTemp_C	30	0	30	0.0
4	DetVel_ms	30	0	30	0.0
5	HeatForm_kJmol	30	1	29	3.3

```
In [3]: # — 1b: Basic descriptive statistics per property —————

stats = df[numeric_cols].describe().T
stats['range'] = stats['max'] - stats['min']
stats['cv%'] = (stats['std'] / stats['mean'] * 100).round(1) # coefficient of variation
display(stats[['count', 'mean', 'std', 'min', 'max', 'range', 'cv%']])

"""
display(stats[['count', 'mean', 'std', 'min', 'max', 'range', 'cv%']]
        .style.format('{:.2f}')
        .background_gradient(subset=['cv%'], cmap='Blues')
        .set_caption('Descriptive Statistics (cv% = coefficient of variation)')
        .set_table_styles([{'selector': 'caption',
                             'props': [('font-size', '14px'), ('font-weight', 'bold')]}])
"""
```

	count	mean	std	min	max	range	cv%
MW	30.0	241.462333	116.148562	80.040	609.210	529.170	48.1
Density	30.0	1.776133	0.162403	1.321	2.044	0.723	9.1
ImpactSens_J	29.0	29.772414	32.077929	0.200	110.000	109.800	107.7
DecompTemp_C	30.0	236.466667	55.822214	155.000	360.000	205.000	23.6
DetVel_ms	30.0	7773.000000	1069.950531	4500.000	9400.000	4900.000	13.8
HeatForm_kJmol	29.0	-67.400000	263.555154	-618.000	460.000	1078.000	-391.0

```
Out[3]: "\ndisplay(stats[['count', 'mean', 'std', 'min', 'max', 'range', 'cv%']])\n
        .style.format('{:.2f}')\n
        .background_gradient(subset=['cv%'], cmap='Blues')\n
        .set_caption('Descriptive Statistics (cv% = coefficient of variation)')\n
        .set_table_styles([{'selector': 'caption',\n
                             'props': [('font-size', '14px'), ('font-weight', 'bold')]}])\n"
```

```
In [4]: # — 1c: Class distribution and completeness per class —————

fig, axes = plt.subplots(1, 2, figsize=(11, 4))

# Pie chart — class distribution
```

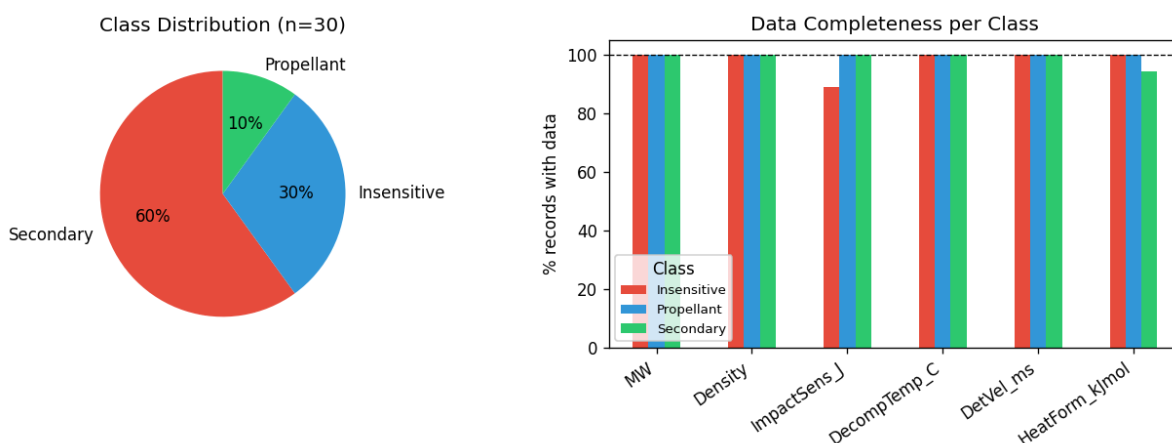
```

class_counts = df['Class'].value_counts()
colors = ['#e74c3c', '#3498db', '#2ecc71', '#f39c12']
axes[0].pie(class_counts, labels=class_counts.index, autopct='%1.0f%%',
            colors=colors, startangle=90)
axes[0].set_title('Class Distribution (n=30)')

# Stacked bar – data completeness per class
comp = df.groupby('Class')[numeric_cols].apply(lambda g: g.notna().mean() * 100)
comp.T.plot(kind='bar', ax=axes[1], color=colors)
axes[1].set_ylabel('% records with data')
axes[1].set_title('Data Completeness per Class')
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=35, ha='right')
axes[1].legend(title='Class', fontsize=8)
axes[1].axhline(100, color='black', linestyle='--', linewidth=0.8)

plt.tight_layout()
plt.show()

```



Block 2 · Structure Standardisation — Canonicalise and Deduplicate

When data comes from multiple papers:

- The same compound may appear under different names or with different SMILES strings
- Canonicalisation ensures every molecule has exactly one standard SMILES
- InChIKey comparison detects duplicate structures reliably

Rule: Always standardise SMILES before calculating any descriptors.

```

In [5]: # — 2a: Validate and canonicalise all SMILES —————
warnings.filterwarnings('ignore')
from rdkit.Chem.inchi import MolToInchi, InchiToInchiKey
warnings.filterwarnings('ignore')
def standardise(smiles):
    """Return (canonical_smiles, inchikey) or (None, None) if invalid."""
    mol = MolFromSmiles(smiles)
    if mol is None:
        return None, None
    canon = MolToSmiles(mol)
    inchi = MolToInchi(mol)
    key = InchiToInchiKey(inchi) if inchi else None
    return canon, key

```

```
df['CanonSMILES'], df['InChIKey'] = zip(*df['SMILES'].map(standardise))

n_invalid = df['CanonSMILES'].isna().sum()
print(f'Invalid SMILES: {n_invalid}')
print(f'Valid structures: {df["CanonSMILES"].notna().sum()}')

# Show original vs canonical for a few compounds
check = df[['Name', 'SMILES', 'CanonSMILES']].head(4).copy()
check['Changed?'] = check['SMILES'] != check['CanonSMILES']

display(check)
"""
display(check.style.hide(axis='index')
        .set_caption('Original vs Canonical SMILES')
        .set_table_styles([{'selector': 'caption',
                             'props': [('font-size', '14px'), ('font-weight', 'bold')]}])
"""
```

Invalid SMILES: 2

Valid structures: 28

```
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] SMILES Parse Error: unclosed ring for input: 'O=[N+](O-))N1CN2CN([N+]
(=O)[O-])CN1[N+](=O)[O-]'
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged; Omitted undefined stereo
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] Can't kekulize mol. Unkekulized atoms: 3 4 8 12 16
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Proton(s) added/removed
[18:34:15] WARNING: Charges were rearranged; Omitted undefined stereo
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged
[18:34:15] WARNING: Charges were rearranged; Omitted undefined stereo
[18:34:15] WARNING: Charges were rearranged
```


	Name	SMILES	CanonSMILES	Changed?
0	TNT	<chem>Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]</chem>	<chem>Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]</chem>	False
1	RDX	<chem>O=[N+](O-)N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1</chem>	<chem>O=[N+](O-)N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1</chem>	False
2	HMX	<chem>O=[N+](O-)N1CN([N+](=O)[O-])CN([N+](=O)[O-])...</chem>	<chem>O=[N+](O-)N1CN([N+](=O)[O-])CN([N+](=O)[O-])...</chem>	False
3	PETN	<chem>C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])...</chem>	<chem>O=[N+](O-)OCC(CO[N+](=O)[O-])(CO[N+](=O)[O-])...</chem>	True

```
Out[5]: "\ndisplay(check.style.hide(axis='index'))\n          .set_caption('Original vs Canon\nical SMILES')\n          .set_table_styles([{'selector':'caption',\n'props':[( 'font-size', '14px'),( 'font-weight', 'bold')]}]))\n"
```

```
In [6]: # — 2b: Detect duplicates using InChIKey —————

# Inject two known duplicates to demonstrate detection
dup_rows = pd.DataFrame([
    ['TNT-dup', 'O=[N+](O-)c1cc([N+](=O)[O-])cc(C)c1[N+](=O)[O-]', # different
    'C7H5N3O6', 227.13, 1.654, 15.0, 300, 6900, -54.4, 'Secondary', None, None],
    ['RDX-copy', 'O=[N+](O-)N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1', # exact dup
    'C3H6N6O6', 222.12, 1.820, 7.4, 210, 8750, 86.3, 'Secondary', None, None],
], columns=df.columns)

# Standardise the injected rows
dup_rows['CanonSMILES'], dup_rows['InChIKey'] = zip(
    *dup_rows['SMILES'].map(standardise))

df_with_dups = pd.concat([df, dup_rows], ignore_index=True)
print(f'Dataset size before dedup: {len(df_with_dups)}')

# Find duplicates
dup_mask = df_with_dups.duplicated(subset='InChIKey', keep='first')
dup_table = df_with_dups[df_with_dups['InChIKey'].duplicated(keep=False)][['Name', '
print(f'Duplicate entries found: {dup_mask.sum()}')
display(dup_table.sort_values('InChIKey'))
"""
display(dup_table.sort_values('InChIKey').style.hide(axis='index')
        .set_caption('Detected Duplicates (same InChIKey)')
        .set_table_styles([{'selector':'caption',
        'props':[( 'font-size', '14px'),( 'font-weight', 'bold')]}]))
"""

# Remove duplicates – keep first occurrence
df = df_with_dups[~dup_mask].reset_index(drop=True)
print(f'Dataset size after dedup: {len(df)}')
```

Dataset size before dedup: 32
Duplicate entries found: 5

[18:34:15] WARNING: Charges were rearranged

[18:34:15] WARNING: Charges were rearranged

	Name	InChIKey	CanonSMILES
22	NG	ASIGVDLTBLZXNC-UHFFFAOYSA-N	<chem>O=[N+][O-]OCC(O)CO[N+](=O)[O-]</chem>
26	MHN	ASIGVDLTBLZXNC-UHFFFAOYSA-N	<chem>O=[N+][O-]OCC(O)CO[N+](=O)[O-]</chem>
14	BTNEU	BVKZIAODKDJPLW-UHFFFAOYSA-N	<chem>O=[N+][O-]c1c([N+](=O)[O-])c([N+](=O)[O-])c(...</chem>
17	NONA	BVKZIAODKDJPLW-UHFFFAOYSA-N	<chem>O=[N+][O-]c1c([N+](=O)[O-])c([N+](=O)[O-])c(...</chem>
13	Nitroguanidine	IDCPFAYURAQKDZ-UHFFFAOYSA-N	<chem>N=C(N)N[N+](=O)[O-]</chem>
27	NQ	IDCPFAYURAQKDZ-UHFFFAOYSA-N	<chem>N=C(N)N[N+](=O)[O-]</chem>
1	RDX	XTFIVUDBNACUBN-UHFFFAOYSA-N	<chem>O=[N+][O-]N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1</chem>
31	RDX-copy	XTFIVUDBNACUBN-UHFFFAOYSA-N	<chem>O=[N+][O-]N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1</chem>
8	CL-20	None	None
18	BTF	None	None

Dataset size after dedup: 27

Block 3 · Explore the Dataset — Distributions and Correlations

Always **visualise your data** before building any model.

Look for: skewed distributions, natural groupings, correlations between properties.

```
In [7]: # — 3a: Property distribution histograms —————

df_clean = df.dropna(subset=['InChIKey']).copy()

fig, axes = plt.subplots(2, 3, figsize=(13, 7))
axes = axes.flatten()

prop_labels = {
    'Density': 'Crystal Density (g/cc)',
    'ImpactSens_J': 'Impact Sensitivity (J)',
    'DecompTemp_C': 'Decomp. Temperature (°C)',
    'DetVel_ms': 'Detonation Velocity (m/s)',
    'HeatForm_kJmol': 'Heat of Formation (kJ/mol)',
    'MW': 'Molecular Weight (g/mol)',
}

class_colors = {'Secondary': '#e74c3c', 'Insensitive': '#3498db',
                'Propellant': '#2ecc71'}

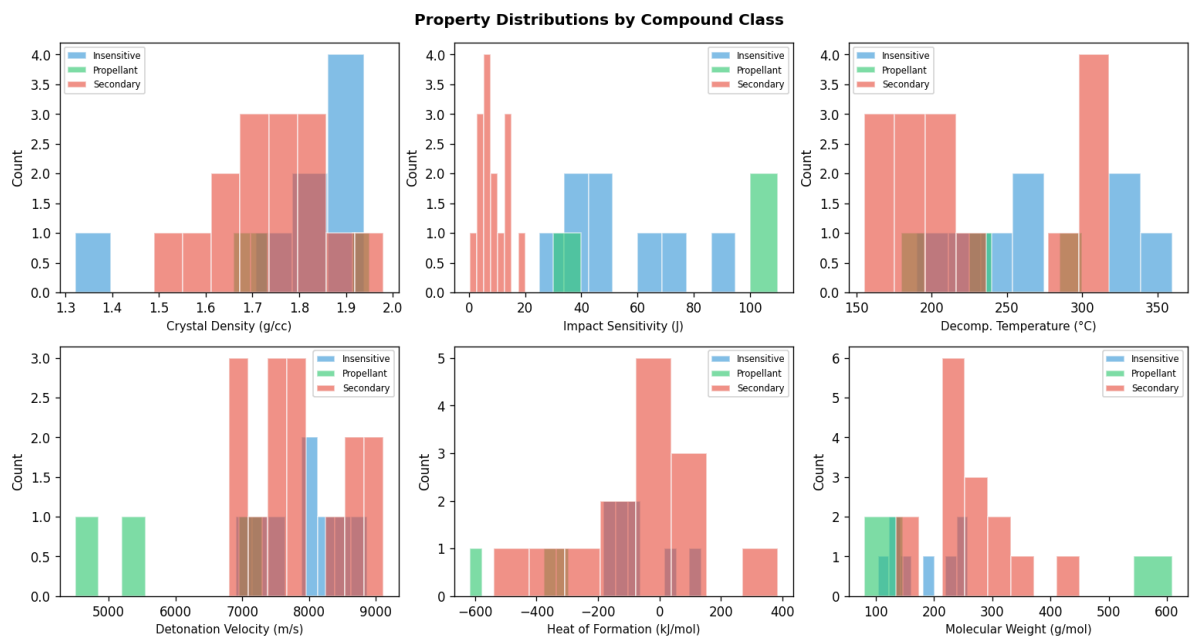
for ax, (col, label) in zip(axes, prop_labels.items()):
    for cls, grp in df_clean.groupby('Class'):
        vals = grp[col].dropna()
        if len(vals) > 1:
            ax.hist(vals, bins=8, alpha=0.6, label=cls,
                    color=class_colors.get(cls, 'grey'), edgecolor='white')
```

```

ax.set_xlabel(label, fontsize=9)
ax.set_ylabel('Count')
ax.legend(fontsize=7)

plt.suptitle('Property Distributions by Compound Class', fontsize=12, fontweight='b')
plt.tight_layout()
plt.show()

```



```

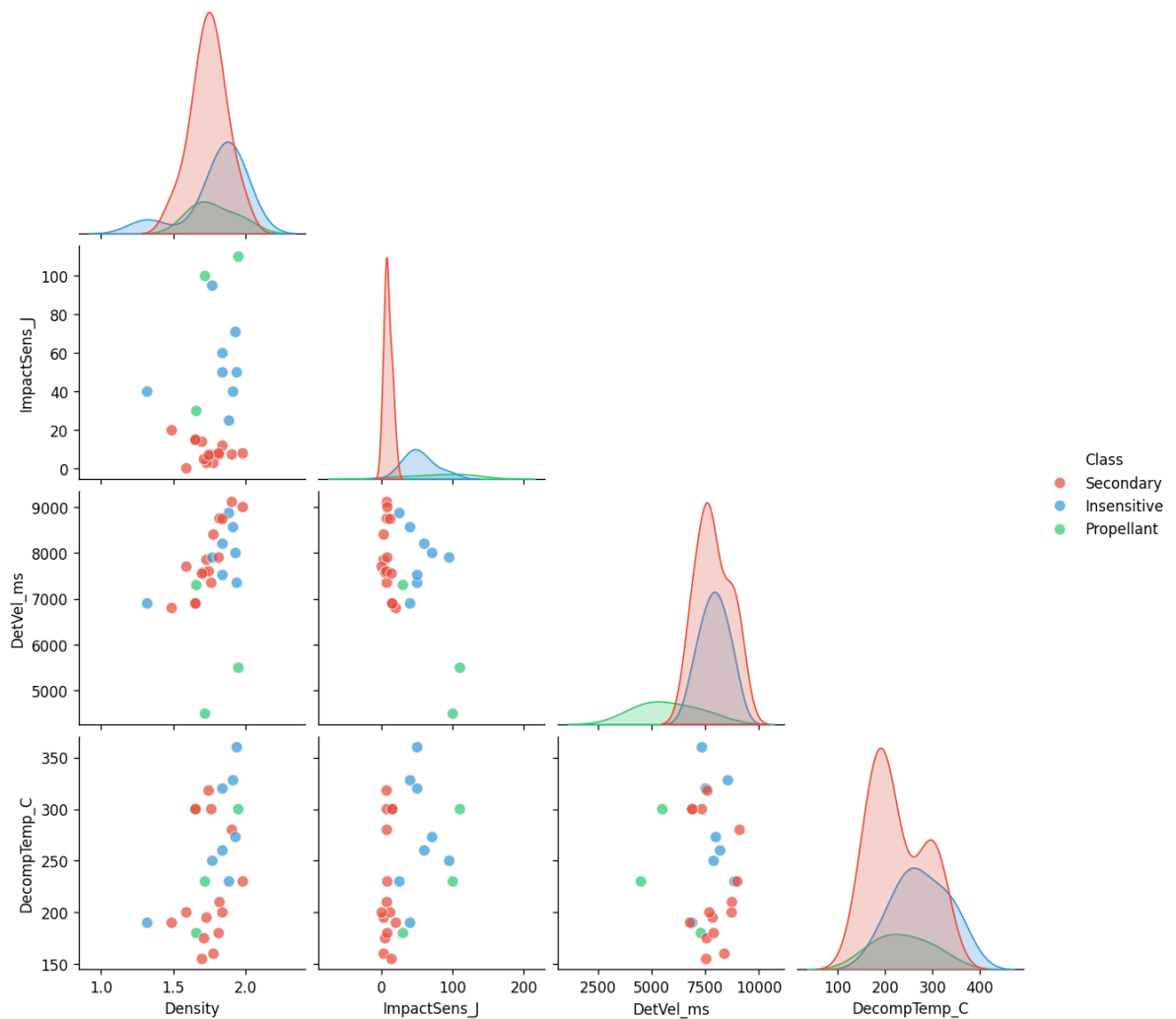
In [8]: # — 3b: Pairplot – density vs sensitivity vs det. velocity —

pair_cols = ['Density', 'ImpactSens_J', 'DetVel_ms', 'DecompTemp_C']
df_pair = df_clean[pair_cols + ['Class']].dropna()

g = sns.pairplot(df_pair, hue='Class',
                 palette={'Secondary': '#e74c3c', 'Insensitive': '#3498db',
                          'Propellant': '#2ecc71'},
                 diag_kind='kde', plot_kws={'alpha': 0.7, 's': 60},
                 corner=True)
g.figure.suptitle('Pairplot of Key Energetic Properties', y=1.01, fontsize=12)
plt.show()
print('Diagonal: KDE of each property')
print('Off-diagonal: scatter between pairs – look for linear or curved trends')

```

Pairplot of Key Energetic Properties



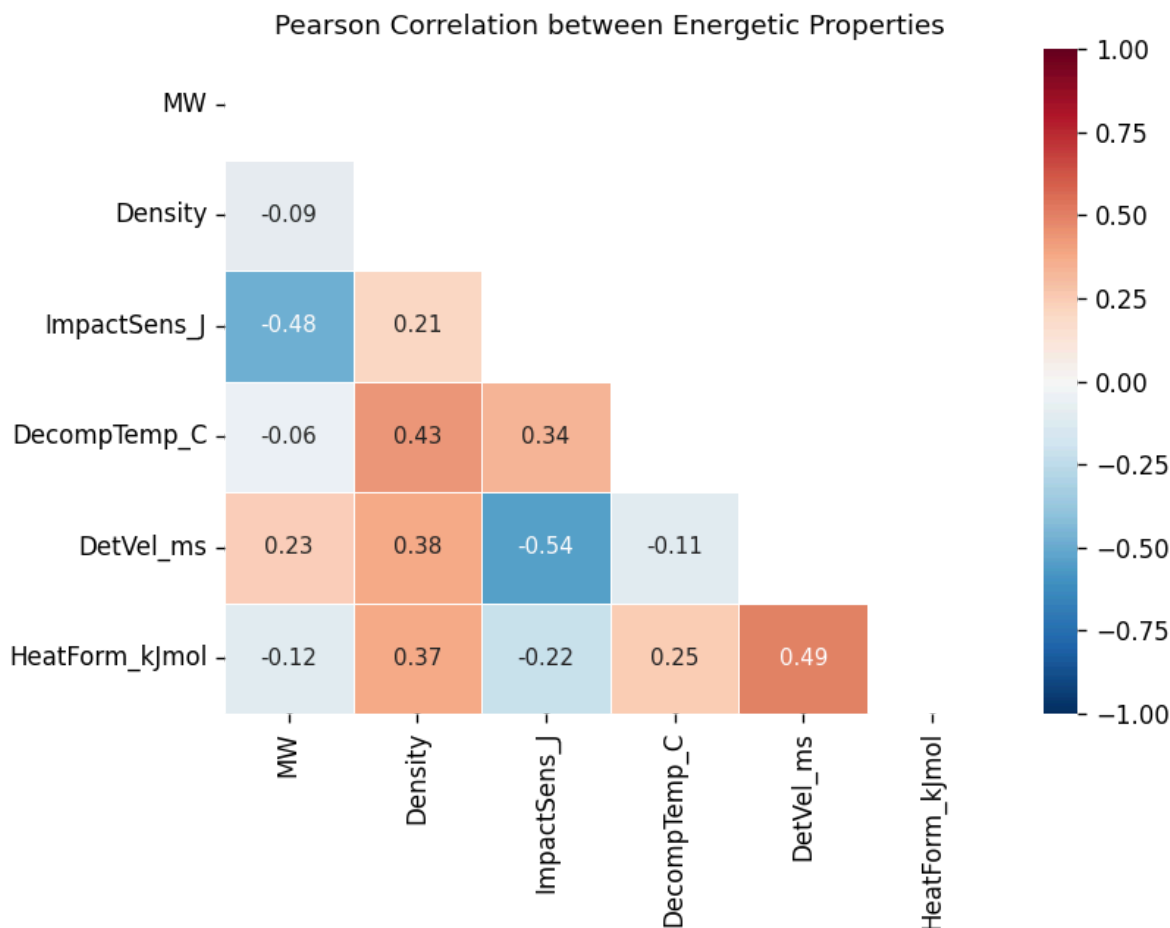
Diagonal: KDE of each property

Off-diagonal: scatter between pairs – look for linear or curved trends

```
In [9]: # — 3c: Correlation heatmap between properties —————

corr = df_clean[numeric_cols].corr()

fig, ax = plt.subplots(figsize=(7, 5.5))
mask = np.triu(np.ones_like(corr, dtype=bool)) # show lower triangle only
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f',
            cmap='RdBu_r', center=0, vmin=-1, vmax=1,
            linewidths=0.5, ax=ax, annot_kws={'size': 9})
ax.set_title('Pearson Correlation between Energetic Properties', fontsize=11)
plt.tight_layout()
plt.show()
print('Strong positive (red) = both increase together')
print('Strong negative (blue) = one increases as the other decreases')
```



Strong positive (red) = both increase together

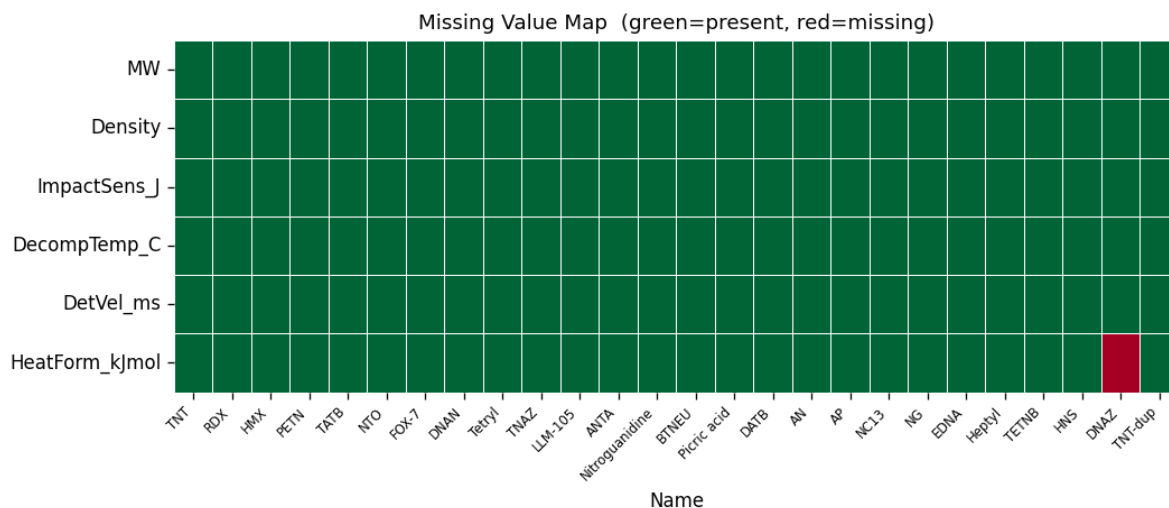
Strong negative (blue) = one increases as the other decreases

Block 4 · Detect and Handle Missing Values

Three strategies depending on how many values are missing:

Strategy	When to use
Drop rows	Target property is missing — cannot use for training
Drop columns	>40% of values missing — descriptor is unreliable
Impute (fill)	<20% missing — replace with mean/median/mode

```
In [10]: # — 4a: Visualise missing value pattern —————
fig, ax = plt.subplots(figsize=(9, 4))
missing_matrix = df_clean[['Name'] + numeric_cols].set_index('Name').isna()
sns.heatmap(missing_matrix.T.astype(int), cmap='RdYlGn_r',
            linewidths=0.3, ax=ax, cbar=False,
            xticklabels=True, yticklabels=True)
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right', fontsize=7)
ax.set_title('Missing Value Map (green=present, red=missing)', fontsize=11)
plt.tight_layout()
plt.show()
```



```
In [11]: # — 4b: Strategy 1 – drop rows where TARGET is missing ———
# Here the target is ImpactSens_J (impact sensitivity)
```

```
target = 'ImpactSens_J'
df_target = df_clean.dropna(subset=[target]).copy()

print(f'Rows before dropping missing target: {len(df_clean)}')
print(f'Rows after dropping missing target: {len(df_target)}')
dropped = df_clean[df_clean[target].isna()][ 'Name' ].tolist()
print(f'Dropped compounds: {dropped}')
```

Rows before dropping missing target: 26

Rows after dropping missing target: 26

Dropped compounds: []

```
In [12]: # — 4c: Strategy 2 – mean/median imputation for features ———
# Impute HeatForm_kJmol (which has some missing values)
```

```
print('Before imputation:')
print(df_target[['Name', 'HeatForm_kJmol']].to_string(index=False))

imputer = SimpleImputer(strategy='median') # 'mean', 'median', or 'most_frequent'
df_target[['HeatForm_kJmol']] = imputer.fit_transform(df_target[['HeatForm_kJmol']])

print(f'\nAfter median imputation: {df_target["HeatForm_kJmol"].isna().sum()} missing')
print(f'Imputed median value used: {imputer.statistics_[0]:.2f} kJ/mol')
```

Before imputation:

Name	HeatForm_kJmol
TNT	-54.4
RDX	86.3
HMX	75.0
PETN	-538.5
TATB	-139.7
NTO	-91.7
FOX-7	-133.8
DNAN	-182.7
Tetryl	18.9
TNAZ	17.0
LLM-105	135.6
ANTA	28.0
Nitroguanidine	-92.5
BTNEU	385.0
Picric acid	-217.8
DATB	-143.5
AN	-365.6
AP	-295.8
NC13	-618.0
NG	-370.9
EDNA	-148.0
Heptyl	-136.0
TETNB	125.0
HNS	22.5
DNAZ	NaN
TNT-dup	-54.4

After median imputation: 0 missing values remain

Imputed median value used: -92.50 kJ/mol

```
In [13]: # — 4d: Strategy 3 – KNN imputation (smarter, uses neighbours) –

from sklearn.impute import KNNImputer

# Reintroduce missing values to demonstrate
df_knn = df_clean[numeric_cols].copy()
n_missing_before = df_knn.isna().sum().sum()

knn_imp = KNNImputer(n_neighbors=3) # use 3 nearest compounds to fill gaps
df_knn_filled = pd.DataFrame(
    knn_imp.fit_transform(df_knn),
    columns=numeric_cols
)

print(f'Missing values before KNN imputation: {n_missing_before}')
print(f'Missing values after KNN imputation: {df_knn_filled.isna().sum().sum()}')
print()
print('KNN imputer fills each gap using the average of the 3')
print('most similar rows in descriptor space – more realistic than mean imputation.'
```

Missing values before KNN imputation: 1

Missing values after KNN imputation: 0

KNN imputer fills each gap using the average of the 3

most similar rows in descriptor space – more realistic than mean imputation.

Block 5 · Detect and Handle Outliers

Outliers can be:

- **Real** — a genuinely extreme compound (do not delete without checking)
- **Errors** — transcription mistakes from papers (should be corrected)

Detection methods covered:

1. Box plots (visual)
2. Z-score (statistical — flags values $> 3\sigma$ from mean)
3. IQR fence (robust — flags values beyond $1.5 \times \text{IQR}$)
4. Isolation Forest (ML-based)

```
In [14]: # — 5a: Box plots – visual outlier detection —————

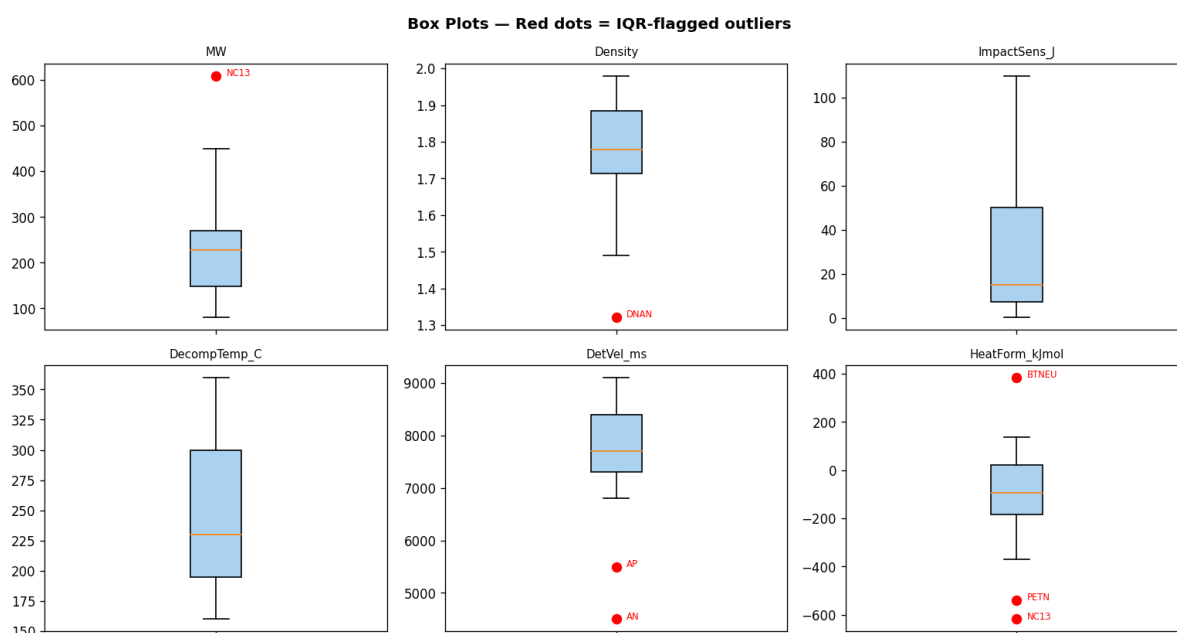
df_plot = df_clean[numeric_cols].dropna()

fig, axes = plt.subplots(2, 3, figsize=(13, 7))
axes = axes.flatten()

for ax, col in zip(axes, numeric_cols):
    q1, q3 = df_plot[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr
    outliers = df_clean[['Name', col]].dropna()
    outliers = outliers[(outliers[col] < lo) | (outliers[col] > hi)]

    ax.boxplot(df_plot[col].dropna(), vert=True, patch_artist=True,
               boxprops=dict(facecolor='#AED6F1'))
    # Overlay outlier labels
    for _, row in outliers.iterrows():
        ax.scatter(1, row[col], color='red', zorder=5, s=50)
        ax.annotate(row['Name'], (1, row[col]),
                    textcoords='offset points', xytext=(8, 0), fontsize=7, color='r')
    ax.set_title(col, fontsize=9)
    ax.set_xticklabels([])

plt.suptitle('Box Plots – Red dots = IQR-flagged outliers', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()
```



```
In [15]: # — 5b: Z-score outlier detection —————
```



```

from scipy import stats as spstats

z_threshold = 2.5 # flag if |z| > 2.5

flagged_rows = []
for col in numeric_cols:
    subset = df_clean[['Name', col]].dropna()
    z = np.abs(spstats.zscore(subset[col]))
    flagged = subset[z > z_threshold]
    for _, row in flagged.iterrows():
        flagged_rows.append({
            'Compound': row['Name'],
            'Property': col,
            'Value': round(row[col], 2),
            '|Z-score|': round(z[z > z_threshold].iloc[
                list(flagged.index).index(row.name)], 2)
        })

df_flagged = pd.DataFrame(flagged_rows)
if df_flagged.empty:
    print('No outliers flagged at |Z| > 2.5')
else:
    display(df_flagged.sort_values('|Z-score|', ascending=False))
    """
        display(df_flagged.sort_values('|Z-score|', ascending=False)
            .style.hide(axis='index')
            .background_gradient(subset=['|Z-score|'], cmap='Reds')
            .set_caption(f'Z-score Outliers (threshold |Z| > {z_threshold})')
            .set_table_styles([{'selector': 'caption',
                                'props': [('font-size', '14px'), ('font-weight', 'bold')]
                                }])
    """

    print()
    print('Action: go back to the original paper to verify each flagged value.')
    print('Do NOT delete automatically – some extremes are chemically real.')

```

	Compound	Property	Value	Z-score
0	NC13	MW	609.21	3.37
2	AN	DetVel_ms	4500.00	3.08
1	DNAN	Density	1.32	3.02

Action: go back to the original paper to verify each flagged value.
Do NOT delete automatically – some extremes are chemically real.

In [16]: # — 5c: Isolation Forest – ML-based multivariate outlier detection

```

from sklearn.ensemble import IsolationForest

df_iso = df_clean[numeric_cols].dropna()
names_iso = df_clean.loc[df_iso.index, 'Name'].values

iso = IsolationForest(contamination=0.1, random_state=42) # expect ~10% outliers
labels = iso.fit_predict(df_iso) # -1 = outlier, +1 = inlier
scores = iso.score_samples(df_iso)

iso_result = pd.DataFrame({
    'Compound': names_iso,
    'IsoScore': scores.round(4),
    'Label': ['Outlier' if l == -1 else 'Inlier' for l in labels]
}).sort_values('IsoScore')

print(f'Isolation Forest results ({sum(labels==-1)} outliers flagged):')

```

```
"""
display(iso_result.style
        .applymap(lambda v: 'background-color:#ffcccc' if v=='Outlier' else '', sub
        .hide(axis='index')
        .set_caption('Isolation Forest Outlier Detection (lower score = more anomal
        .set_table_styles([{'selector':'caption',
                            'props':[('font-size','14px'),('font-weight','bold')]}])
"""
display(iso_result)
```

Isolation Forest results (3 outliers flagged):

	Compound	IsoScore	Label
18	NC13	-0.5977	Outlier
16	AN	-0.5975	Outlier
17	AP	-0.5691	Outlier
13	BTNEU	-0.5637	Inlier
3	PETN	-0.5159	Inlier
7	DNAN	-0.5060	Inlier
23	HNS	-0.4895	Inlier
4	TATB	-0.4785	Inlier
10	LLM-105	-0.4758	Inlier
12	Nitroguanidine	-0.4722	Inlier
19	NG	-0.4682	Inlier
2	HMX	-0.4653	Inlier
21	Heptyl	-0.4626	Inlier
11	ANTA	-0.4569	Inlier
5	NTO	-0.4513	Inlier
6	FOX-7	-0.4463	Inlier
20	EDNA	-0.4358	Inlier
15	DATB	-0.4248	Inlier
14	Picric acid	-0.4203	Inlier
0	TNT	-0.4180	Inlier
24	TNT-dup	-0.4180	Inlier
22	TETNB	-0.4168	Inlier
1	RDX	-0.4159	Inlier
8	Tetryl	-0.4158	Inlier
9	TNAZ	-0.4102	Inlier

Block 6 · Descriptor Calculation and Feature Matrix Assembly

Now we calculate **molecular descriptors** for every compound in the dataset and assemble the full feature matrix **X** paired with the target vector **y**.

We combine:

- **Zero-dimensional** descriptors (formula-based)
- **Two-dimensional** RDKit descriptors (topology)
- **Morgan fingerprint** bits (structural patterns)

```
In [17]: # — 6a: Calculate descriptors for all valid compounds —————

fpngen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=512)

def parse_formula(formula):
    counts = {}
    for elem in ['C', 'H', 'N', 'O']:
        m = re.search(rf'{elem}(\d*)', str(formula))
        counts[elem] = int(m.group(1)) if m and m.group(1) else (1 if m else 0)
    return counts

def calc_all_descriptors(name, smiles, formula, mw):
    """Returns a dict of descriptors for one compound."""
    mol = MolFromSmiles(smiles)
    if mol is None:
        return None

    e = parse_formula(formula)
    ob = round((1600/mw)*(2*e['C']+e['H']/2-e['O'])), 2) if mw else None

    # 0D
    d = {
        'Name': name,
        'MW': round(mw, 2),
        'nC': e['C'], 'nH': e['H'], 'nN': e['N'], 'nO': e['O'],
        'nNO2': smiles.count('[N+](=O)[O-]'),
        'NC_ratio': round(e['N']/e['C'], 3) if e['C'] else 0,
        'OB_pct': ob,
    }

    # 2D RDKit
    d.update({
        'TPSA': round(Descriptors.TPSA(mol), 2),
        'LogP': round(Descriptors.MolLogP(mol), 3),
        'HBD': rdMolDescriptors.CalcNumHBD(mol),
        'HBA': rdMolDescriptors.CalcNumHBA(mol),
        'RotBonds': rdMolDescriptors.CalcNumRotatableBonds(mol),
        'Rings': rdMolDescriptors.CalcNumRings(mol),
        'ArRings': rdMolDescriptors.CalcNumAromaticRings(mol),
        'HeavyAtoms': mol.GetNumHeavyAtoms(),
        'FrCSP3': round(Descriptors.FractionCSP3(mol), 4),
        'BalabanJ': round(Descriptors.BalabanJ(mol), 4),
        'Chi0': round(Descriptors.Chi0(mol), 4),
        'Chi1': round(Descriptors.Chi1(mol), 4),
        'Kappa1': round(Descriptors.Kappa1(mol), 4),
    })

    # Morgan fingerprint bits (512-bit)
    fp = fpngen.GetFingerprint(mol)
    arr = np.zeros(512, dtype=np.uint8)
    DataStructs.ConvertToNumpyArray(fp, arr)
    for i, bit in enumerate(arr):
        d[f'FP{i}'] = int(bit)
```

```

return d

# Calculate for all compounds with valid SMILES
desc_rows = []
for _, row in df_target.iterrows():
    result = calc_all_descriptors(
        row['Name'], row['CanonSMILES'], row['Formula'], row['MW'])
    if result:
        result['ImpactSens_J'] = row['ImpactSens_J']
        result['Density'] = row['Density']
        result['DetVel_ms'] = row['DetVel_ms']
        result['Class'] = row['Class']
        desc_rows.append(result)

df_desc = pd.DataFrame(desc_rows)
print(f'Feature matrix assembled: {df_desc.shape}')
print(f' {df_desc.shape[0]} compounds x {df_desc.shape[1]} columns')
print(f' (includes Name, targets, class + {512+13+9} descriptor columns)')

```

Feature matrix assembled: (26, 538)
 26 compounds x 538 columns
 (includes Name, targets, class + 534 descriptor columns)

In [18]: # — 6b: Inspect descriptor matrix —————

```

# Non-fingerprint descriptors only (easier to read)
meta_cols = ['Name', 'ImpactSens_J', 'Density', 'DetVel_ms', 'Class']
struct_cols = ['MW', 'nC', 'nH', 'nN', 'nO', 'nNO2', 'NC_ratio', 'OB_pct',
               'TPSA', 'LogP', 'HBD', 'HBA', 'RotBonds', 'Rings', 'ArRings',
               'HeavyAtoms', 'FrCSP3', 'BalabanJ', 'Chi0', 'Chi1', 'Kappa1']
display(df_desc[meta_cols + struct_cols].head(6))

"""
display(df_desc[meta_cols + struct_cols].head(6)
        .style.hide(axis='index')
        .format({c: '{:.2f}' for c in struct_cols if c not in ['nC', 'nH', 'nN', 'nO',
        .background_gradient(subset=['nNO2', 'OB_pct', 'TPSA'], cmap='YlOrRd')
        .set_caption('Descriptor Matrix – first 6 rows, structural descriptors only')
        .set_table_styles([{'selector': 'caption',
        'props': [('font-size', '14px'), ('font-weight', 'bold')]}])
"""

```

	Name	ImpactSens_J	Density	DetVel_ms	Class	MW	nC	nH	nN	nO	...	HBA	RotB
0	TNT	15.0	1.654	6900	Secondary	227.13	7	5	3	6	...	6	
1	RDX	7.4	1.820	8750	Secondary	222.12	3	6	6	6	...	6	
2	HMX	7.4	1.905	9110	Secondary	296.16	4	8	8	8	...	8	
3	PETN	3.0	1.778	8400	Secondary	316.14	5	8	4	12	...	12	
4	TATB	50.0	1.939	7350	Insensitive	258.15	6	6	6	6	...	9	
5	NTO	71.0	1.930	8000	Insensitive	130.06	2	2	4	3	...	4	

6 rows x 26 columns

```
Out[18]: "\ndisplay(df_desc[meta_cols + struct_cols].head(6)\n          .style.hide(axis='index'))\n          .format({c: '{:.2f}' for c in struct_cols if c not in ['nC','nH','nN','nO','nNO2','Rings','ArRings','HBD','HBA','RotBonds','HeavyAtoms']})\n          .background_gradient(subset=['nNO2','OB_pct','TPSA'], cmap='YlOrRd')\n          .set_caption('Descriptor Matrix – first 6 rows, structural descriptors only')\n          .set_table_styles([{'selector': 'caption',\n                                'props':\n                                [ ('font-size', '14px'), ('font-weight', 'bold') ]}]))\n"
```

Block 7 · Feature Preprocessing — Filter, Correlation, Scaling

Raw descriptor matrices contain:

- **Constant or near-constant columns** (useless — remove)
- **Highly correlated pairs** (redundant — remove one)
- **Different scales** (unfair advantage — normalise)

This block walks through each step.

```
In [19]: # — 7a: Separate feature matrix X and target y —————

target_col = 'ImpactSens_J'          # property we want to predict
drop_cols  = ['Name', 'ImpactSens_J', 'Density', 'DetVel_ms', 'Class']

X_raw = df_desc.drop(columns=drop_cols).astype(float)
y      = df_desc[target_col].astype(float)
names  = df_desc['Name'].values

print(f'X shape (raw): {X_raw.shape} — {X_raw.shape[1]} features')
print(f'y shape:      {y.shape}')
```

```
X shape (raw): (26, 533) — 533 features
y shape:      (26,)
```

```
In [20]: # — 7b: Step 1 – remove near-zero variance columns —————

vt = VarianceThreshold(threshold=0.01) # remove columns with variance < 0.01
X_vt = vt.fit_transform(X_raw)

removed_var = X_raw.shape[1] - X_vt.shape[1]
kept_cols   = X_raw.columns[vt.get_support()].tolist()

print(f'Before variance filter: {X_raw.shape[1]} features')
print(f'Removed (near-zero var): {removed_var}')
print(f'Remaining:                {X_vt.shape[1]}')

X_df = pd.DataFrame(X_vt, columns=kept_cols)
```

```
Before variance filter: 533 features
Removed (near-zero var): 331
Remaining:                202
```

```
In [21]: # — 7c: Step 2 – remove highly correlated descriptor pairs ———

def remove_high_corr(df_feat, threshold=0.90):
    """Remove one feature from each pair with |correlation| > threshold."""
    corr_mat = df_feat.corr().abs()
    upper    = corr_mat.where(np.triu(np.ones(corr_mat.shape), k=1).astype(bool))
```

```
to_drop = [col for col in upper.columns if any(upper[col] > threshold)]
return df_feat.drop(columns=to_drop), to_drop
```

```
X_uncorr, dropped_corr = remove_high_corr(X_df, threshold=0.90)
```

```
print(f'After variance filter: {X_df.shape[1]} features')
print(f'Removed (corr > 0.90): {len(dropped_corr)}')
print(f'Remaining after both: {X_uncorr.shape[1]}')
if dropped_corr:
    print(f'Example dropped: {dropped_corr[:5]}')
```

After variance filter: 202 features

Removed (corr > 0.90): 118

Remaining after both: 84

Example dropped: ['nO', 'TPSA', 'HBA', 'RotBonds', 'HeavyAtoms']

```
In [22]: # — 7d: Step 3 – scale features —————

# StandardScaler: mean=0, std=1 (best for SVMs, neural networks)
# MinMaxScaler: range [0, 1] (good for tree models, visualisation)

scaler_std = StandardScaler()
scaler_mm = MinMaxScaler()

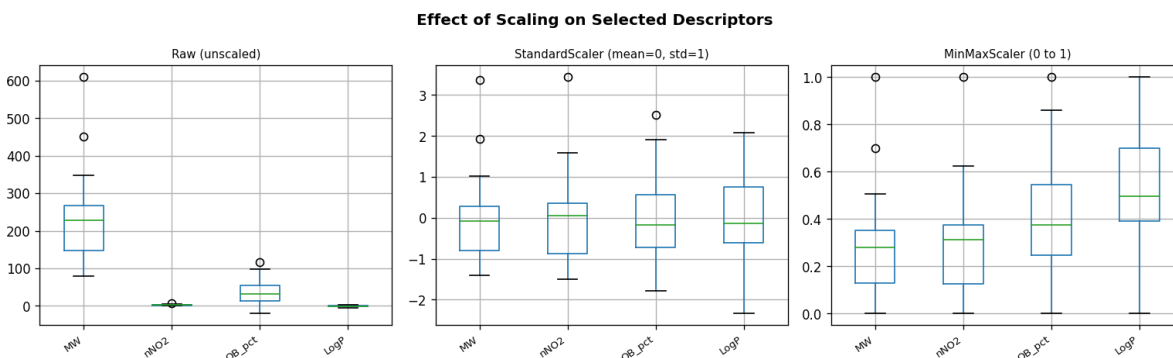
X_std = pd.DataFrame(scaler_std.fit_transform(X_uncorr),
                     columns=X_uncorr.columns)
X_mm = pd.DataFrame(scaler_mm.fit_transform(X_uncorr),
                   columns=X_uncorr.columns)

# Compare raw vs scaled for a few key descriptors
compare_cols = [c for c in ['MW', 'TPSA', 'nNO2', 'OB_pct', 'LogP'] if c in X_uncorr.columns]

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
for ax, (data, title) in zip(axes, [
    (X_uncorr[compare_cols], 'Raw (unscaled)'),
    (X_std[compare_cols], 'StandardScaler (mean=0, std=1)'),
    (X_mm[compare_cols], 'MinMaxScaler (0 to 1)'),
]):
    data.boxplot(ax=ax)
    ax.set_title(title, fontsize=9)
    ax.set_xticklabels(ax.get_xticklabels(), rotation=35, ha='right', fontsize=8)

plt.suptitle('Effect of Scaling on Selected Descriptors', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()

print(f'Final clean feature matrix: {X_std.shape}')
```



Final clean feature matrix: (26, 84)

Block 8 · Train / Test Splitting — Random and Scaffold-Based

The golden rule: never evaluate a model on data it was trained on.

Two splitting strategies:

Method	When to use	Pros / Cons
Random split	Quick baseline	Fast; similar molecules may end up in both sets
Scaffold split	Realistic evaluation	Similar molecules stay together; harder test

```
In [23]: # — 8a: Simple random split (70 / 30) —————

X_train_r, X_test_r, y_train_r, y_test_r, names_train_r, names_test_r = \
    train_test_split(X_std, y, names,
                    test_size=0.30, random_state=42)

print('Random split (70/30)')
print(f' Training set: {len(X_train_r)} compounds — {list(names_train_r)}')
print(f' Test set:      {len(X_test_r)}  compounds — {list(names_test_r)}')

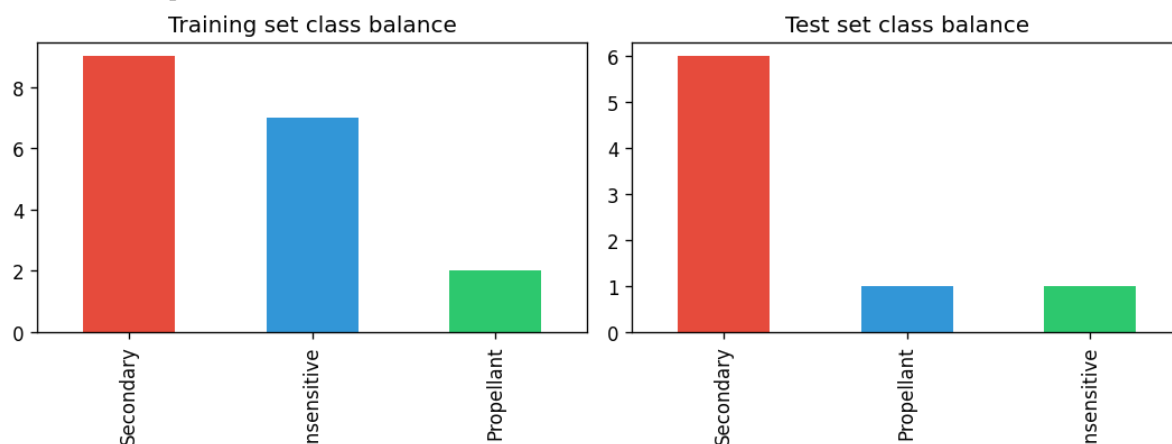
# Class balance check
cls_train = df_desc.set_index('Name').loc[names_train_r, 'Class'].value_counts()
cls_test  = df_desc.set_index('Name').loc[names_test_r, 'Class'].value_counts()

fig, axes = plt.subplots(1, 2, figsize=(9, 3.5))
cls_train.plot(kind='bar', ax=axes[0], color=['#e74c3c', '#3498db', '#2ecc71'])
axes[0].set_title('Training set class balance'); axes[0].set_xlabel('')
cls_test.plot(kind='bar', ax=axes[1], color=['#e74c3c', '#3498db', '#2ecc71'])
axes[1].set_title('Test set class balance'); axes[1].set_xlabel('')
plt.tight_layout(); plt.show()
```

Random split (70/30)

Training set: 18 compounds — ['HNS', 'NTO', 'HMX', 'Nitroguanidine', 'DATB', 'PE TN', 'TATB', 'Heptyl', 'AP', 'TETNB', 'NC13', 'TNT-dup', 'EDNA', 'DNAN', 'LLM-10 5', 'Picric acid', 'NG', 'FOX-7']

Test set: 8 compounds — ['Tetryl', 'AN', 'TNT', 'DNAZ', 'ANTA', 'TNAZ', 'BT NEU', 'RDX']



```
In [24]: # — 8b: Scaffold-based split (keeps similar structures together)

def get_scaffold(smiles):
    """Return Murcko scaffold SMILES, or empty string for acyclics."""
    mol = MolFromSmiles(smiles)
    if mol is None:
        return ''
```

```

try:
    scaffold = MurckoScaffold.GetScaffoldForMol(mol)
    return MolToSmiles(scaffold)
except Exception:
    return ''

# Assign scaffold to each compound
df_scaffold = df_raw[['Name', 'SMILES']].copy()
df_scaffold.columns = ['Name', 'SMILES']
df_scaffold['Scaffold'] = df_scaffold['SMILES'].map(get_scaffold)

print('Scaffolds assigned:')

display(df_scaffold)

"""
display(df_scaffold.style.hide(axis='index')
        .set_caption('Murcko Scaffolds (empty = no ring system)')
        .set_table_styles([{'selector': 'caption',
                            'props': [('font-size', '14px'), ('font-weight', 'bold')]}])
"""

```

Scaffolds assigned:

```

[18:34:24] SMILES Parse Error: unclosed ring for input: 'O=[N+]([O-])N1CN2CN([N+]
(=O)[O-])CN1[N+](=O)[O-]'
[18:34:24] Can't kekulize mol.  Unkekulized atoms: 3 4 8 12 16

```


	Name	SMILES	Scaffold
0	TNT	<chem>Cc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]</chem>	c1ccccc1
1	RDX	<chem>O=[N+](O-)[N1CN([N+](=O)[O-])CN([N+](=O)[O-])C1]</chem>	C1NCNCN1
2	HMX	<chem>O=[N+](O-)[N1CN([N+](=O)[O-])CN([N+](=O)[O-])...</chem>	C1NCNCNCN1
3	PETN	<chem>C(CO[N+](=O)[O-])(CO[N+](=O)[O-])(CO[N+](=O)[O-])...</chem>	
4	TATB	<chem>Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])c(N)c1[N+](=O)[O-]</chem>	c1ccccc1
5	NTO	<chem>O=[N+](O-)[c1n[nH]c(=O)[nH]1]</chem>	O=c1[nH]cn[nH]1
6	FOX-7	<chem>NC(=C([N+](=O)[O-])[N+](=O)[O-])N</chem>	
7	DNAN	<chem>COc1ccc([N+](=O)[O-])cc1[N+](=O)[O-]</chem>	c1ccccc1
8	CL-20	<chem>O=[N+](O-)[N1CN2CN([N+](=O)[O-])CN1[N+](=O)[O-]]</chem>	
9	Tetryl	<chem>CN([N+](=O)[O-])c1c([N+](=O)[O-])cc([N+](=O)[O-])cc1</chem>	c1ccccc1
10	TNAZ	<chem>O=[N+](O-)[C1(CN([N+](=O)[O-])C1[N+](=O)[O-])...</chem>	C1CNC1
11	LLM-105	<chem>Nc1nc2c([N+](=O)[O-])n[nH]c2c([N+](=O)[O-])n1</chem>	c1ncc2[nH]ncc2n1
12	ANTA	<chem>Nc1nnc([N+](=O)[O-])[nH]1</chem>	c1nnc[nH]1
13	Nitroguanidine	<chem>NC(=N)N[N+](=O)[O-]</chem>	
14	BTNEU	<chem>O=[N+](O-)[c1c([N+](=O)[O-])c([N+](=O)[O-])c(...</chem>	c1ccccc1
15	Picric acid	<chem>Oc1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]</chem>	c1ccccc1
16	DATB	<chem>Nc1c([N+](=O)[O-])c(N)c([N+](=O)[O-])cc1[N+](=O)[O-]</chem>	c1ccccc1
17	NONA	<chem>O=[N+](O-)[c1c([N+](=O)[O-])c([N+](=O)[O-])c(...</chem>	c1ccccc1
18	BTF	<chem>O=[N+](O-)[c1c([N+](=O)[O-])c([N+](=O)[O-])c(...</chem>	
19	AN	<chem>N.[N+](=O)([O-])[O-]</chem>	
20	AP	<chem>[NH4+].[O-]Cl(=O)(=O)=O</chem>	
21	NC13	<chem>OCC1OC(OC2C(COC(=O)[N+](=O)[O-])OC(OC3C(COC(=O)[N+](=O)[O-])OC3)C1</chem>	C1COCC(OC2CCCC(OC3CCCCO3)CO2)C1
22	NG	<chem>O[C@@H](CO[N+](=O)[O-])CO[N+](=O)[O-]</chem>	
23	EDNA	<chem>O=[N+](O-)[NCC[N+](=O)[O-]]</chem>	
24	Heptyl	<chem>CC(C)(C)c1c([N+](=O)[O-])cc([N+](=O)[O-])cc1[N+](=O)[O-]</chem>	c1ccccc1
25	TETNB	<chem>O=[N+](O-)[c1c([N+](=O)[O-])c([N+](=O)[O-])cc...</chem>	c1ccccc1

	Name	SMILES	Scaffold
26	MHN	<chem>OC(CO[N+](=O)[O-])CO[N+](=O)[O-]</chem>	
27	NQ	<chem>NC(=N)N[N+](=O)[O-]</chem>	
28	HNS	<chem>O=[N+](O-)[c1ccc(C=Cc2ccc([N+](=O)[O-])cc2[N+](=O)[O-])cc1]</chem>	<chem>C(=Cc1cccc1)c1cccc1</chem>
29	DNAZ	<chem>O=[N+](O-)[N1CC([N+](=O)[O-])C1]</chem>	<chem>C1CNC1</chem>

```
Out[24]: "\ndisplay(df_scaffold.style.hide(axis='index')\n          .set_caption('Murcko Scaffolds (empty = no ring system)')\n          .set_table_styles([{'selector':'caption',\n                                'props':[( 'font-size','14px'),('font-weight','bold')]}]))\n"
```

```
In [25]: # — 8c: Assign train/test by scaffold (no scaffold shared) —

import random
random.seed(42)

scaffold_groups = df_scaffold.groupby('Scaffold')['Name'].apply(list).to_dict()
scaffold_list = list(scaffold_groups.values())
random.shuffle(scaffold_list)

test_size_target = 0.30
total = len(df_scaffold)
test_names, train_names = [], []

for group in scaffold_list:
    if len(test_names) / (total + 1e-9) < test_size_target:
        test_names.extend(group)
    else:
        train_names.extend(group)

# Remaining go to train
assigned = set(test_names + train_names)
for n in df_scaffold['Name']:
    if n not in assigned:
        train_names.append(n)

print(f'Scaffold split ({len(train_names)} train / {len(test_names)} test)')
print(f'  Train: {train_names}')
print(f'  Test: {test_names}')

idx_map = df_desc.set_index('Name').index
train_idx = [i for i, n in enumerate(df_desc['Name']) if n in train_names]
test_idx = [i for i, n in enumerate(df_desc['Name']) if n in test_names]

X_train_s = X_std.iloc[train_idx]
X_test_s = X_std.iloc[test_idx]
y_train_s = y.iloc[train_idx]
y_test_s = y.iloc[test_idx]

print(f'\nActual split ratio – train: {len(X_train_s)/len(X_std):.1%}, '
      f'test: {len(X_test_s)/len(X_std):.1%}')

Scaffold split (20 train / 10 test)
  Train: ['NC13', 'TNAZ', 'DNAZ', 'LLM-105', 'HMX', 'NTO', 'ANTA', 'RDX', 'PETN',
'FOX-7', 'CL-20', 'Nitroguanidine', 'BTF', 'AN', 'AP', 'NG', 'EDNA', 'MHN', 'NQ',
'HNS']
  Test: ['TNT', 'TATB', 'DNAN', 'Tetryl', 'BTNEU', 'Picric acid', 'DATB', 'NONA',
'Heptyl', 'TETNB']

Actual split ratio – train: 61.5%, test: 34.6%
```

Block 9 · Full Pipeline — SMILES to ML-Ready Matrix

This block assembles **everything from Blocks 0–8 into one clean, reusable function**.

Give it a list of (name, SMILES, target_value) tuples and it returns train/test arrays ready for scikit-learn.

This is what you would use as your **starting point** for any new dataset.

```
In [26]: # — 9a: Complete pipeline function —————

def build_em_dataset(records, target_key='ImpactSens_J',
                    fp_bits=512, corr_threshold=0.90,
                    var_threshold=0.01, test_size=0.30,
                    random_state=42, scale='standard'):
    """
    records : list of dicts with keys Name, SMILES, Formula, MW, <target_key>, ...
    Returns : X_train, X_test, y_train, y_test, feature_names, scaler
    """
    fpngen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=fp_bits)

    # — Step 1: validate and canonicalise —————
    valid = []
    for rec in records:
        mol = MolFromSmiles(rec['SMILES'])
        if mol is None or rec.get(target_key) is None:
            continue
        rec['CanonSMILES'] = MolToSmiles(mol)
        rec['mol'] = mol
        valid.append(rec)
    print(f'[1] Valid records with target: {len(valid)}')

    # — Step 2: deduplicate by InChIKey —————
    seen, deduped = set(), []
    for rec in valid:
        inchi = MolToInchi(rec['mol'])
        key = InchiToInchiKey(inchi) if inchi else rec['CanonSMILES']
        if key not in seen:
            seen.add(key)
            deduped.append(rec)
    print(f'[2] After deduplication: {len(deduped)}')

    # — Step 3: calculate descriptors —————
    rows, targets, names_out = [], [], []
    for rec in deduped:
        mol = rec['mol']
        e = parse_formula(rec.get('Formula', ''))
        mw = float(rec.get('MW', Descriptors.MolWt(mol)))
        ob = round((1600/mw)*(2*e['C']+e['H']/2-e['O']), 2)

        d = {
            'nNO2': rec['CanonSMILES'].count('[N+](=O)[O-]'),
            'nN': e['N'], 'nO': e['O'], 'nC': e['C'],
            'MW': mw, 'OB_pct': ob,
            'TPSA': Descriptors.TPSA(mol),
            'LogP': Descriptors.MolLogP(mol),
            'HBD': rdMolDescriptors.CalcNumHBD(mol),
            'HBA': rdMolDescriptors.CalcNumHBA(mol),
```

```

        'Rings': rdMolDescriptors.CalcNumRings(mol),
        'FrCSP3': Descriptors.FractionCSP3(mol),
        'Chi0': Descriptors.Chi0(mol),
        'Kappa1': Descriptors.Kappa1(mol),
    }
    fp = fpgen.GetFingerprint(mol)
    arr = np.zeros(fp_bits, dtype=np.uint8)
    DataStructs.ConvertToNumpyArray(fp, arr)
    for i, bit in enumerate(arr):
        d[f'FP{i}'] = int(bit)

    rows.append(d)
    targets.append(float(rec[target_key]))
    names_out.append(rec['Name'])

X_df = pd.DataFrame(rows)
y_arr = np.array(targets)
print(f'[3] Descriptor matrix: {X_df.shape}')

# — Step 4: variance filter —————
vt = VarianceThreshold(threshold=var_threshold)
X_vt = vt.fit_transform(X_df)
feat_names = X_df.columns[vt.get_support()].tolist()
print(f'[4] After variance filter: {len(feat_names)} features')

# — Step 5: correlation filter —————
X_tmp = pd.DataFrame(X_vt, columns=feat_names)
X_unc, dropped = remove_high_corr(X_tmp, corr_threshold)
print(f'[5] After correlation filter: {X_unc.shape[1]} features')

# — Step 6: scale —————
scaler = StandardScaler() if scale == 'standard' else MinMaxScaler()
X_sc = scaler.fit_transform(X_unc)

# — Step 7: train/test split —————
X_tr, X_te, y_tr, y_te, n_tr, n_te = train_test_split(
    X_sc, y_arr, names_out,
    test_size=test_size, random_state=random_state
)
print(f'[6] Train: {len(X_tr)}, Test: {len(X_te)}')
print(f'    Train compounds: {list(n_tr)}')
print(f'    Test compounds: {list(n_te)}')

return X_tr, X_te, y_tr, y_te, X_unc.columns.tolist(), scaler

print('Pipeline function defined. Run Block 9b to execute it.')

```

Pipeline function defined. Run Block 9b to execute it.

```

In [27]: # — 9b: Run the pipeline on our dataset —————

# Convert df_raw to list of dicts
records_in = df_raw.rename(columns={
    'ImpactSens_J': 'ImpactSens_J',
    'SMILES': 'SMILES',
    'Name': 'Name',
    'Formula': 'Formula',
    'MW': 'MW',
}).to_dict(orient='records')

X_train, X_test, y_train, y_test, feature_names, scaler = build_em_dataset(
    records_in,
    target_key='ImpactSens_J',
    fp_bits=256,

```

```
corr_threshold=0.90,  
test_size=0.30  
)  
  
print(f'\nFinal arrays ready for ML:')  
print(f' X_train: {X_train.shape} y_train: {y_train.shape}')  
print(f' X_test: {X_test.shape} y_test: {y_test.shape}')  
print(f' Feature names (first 10): {feature_names[:10]}')
```

```
[18:34:25] SMILES Parse Error: unclosed ring for input: 'O=[N+](O-)]N1CN2CN([N+]  
(=O)[O-])CN1[N+](=O)[O-]'  
[18:34:25] Can't kekulize mol. Unkekulized atoms: 3 4 8 12 16  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged; Omitted undefined stereo  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Charges were rearranged  
  
[18:34:25] WARNING: Proton(s) added/removed  
  
[18:34:25] WARNING: Charges were rearranged; Omitted undefined stereo
```

```
[1] Valid records with target: 28
[2] After deduplication:      25
[3] Descriptor matrix:      (25, 270)
[4] After variance filter:   163 features
[5] After correlation filter: 88 features
[6] Train: 17, Test: 8
    Train compounds: ['TETNB', 'NTO', 'HMX', 'Nitroguanidine', 'DATB', 'PETN', 'TA
TB', 'EDNA', 'AP', 'Heptyl', 'NC13', 'DNAZ', 'DNAN', 'LLM-105', 'Picric acid', 'N
G', 'FOX-7']
    Test compounds: ['Tetryl', 'AN', 'TNT', 'HNS', 'ANTA', 'TNAZ', 'BTNEU', 'RD
X']
```

Final arrays ready for ML:

```
X_train: (17, 88) y_train: (17,)
X_test:  (8, 88)  y_test:  (8,)
Feature names (first 10): ['nNO2', 'nN', 'nO', 'nC', 'OB_pct', 'LogP', 'HBD', 'R
ings', 'FrCSP3', 'FP0']
```

```
[18:34:25] WARNING: Charges were rearranged
[18:34:25] WARNING: Charges were rearranged
[18:34:25] WARNING: Charges were rearranged
[18:34:25] WARNING: Charges were rearranged
[18:34:25] WARNING: Charges were rearranged
[18:34:25] WARNING: Charges were rearranged
[18:34:25] WARNING: Charges were rearranged; Omitted undefined stereo
[18:34:25] WARNING: Charges were rearranged
```

```
In [28]: # — 9c: Quick sanity check – train a simple model —————
# (Full modelling is covered in Chapter 4)

from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_absolute_error

rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

y_pred_train = rf.predict(X_train)
y_pred_test  = rf.predict(X_test)

r2_train = r2_score(y_train, y_pred_train)
r2_test  = r2_score(y_test, y_pred_test)
mae_test = mean_absolute_error(y_test, y_pred_test)

print(f'Sanity check – Random Forest on cleaned dataset:')
print(f'  R2 training : {r2_train:.3f}')
print(f'  R2 test    : {r2_test:.3f}')
print(f'  MAE test    : {mae_test:.2f} J')
print()
print('Training R2 >> Test R2 means some overfitting – expected with n=20.')
print('Larger datasets and cross-validation address this (see Chapter 4).')

# Observed vs predicted plot
fig, ax = plt.subplots(figsize=(5, 4.5))
ax.scatter(y_train, y_pred_train, color='steelblue', alpha=0.8,
           label=f'Train R2= {r2_train:.2f}', s=60)
ax.scatter(y_test, y_pred_test, color='tomato', alpha=0.9,
           label=f'Test R2= {r2_test:.2f}', s=80, marker='D')
```

```

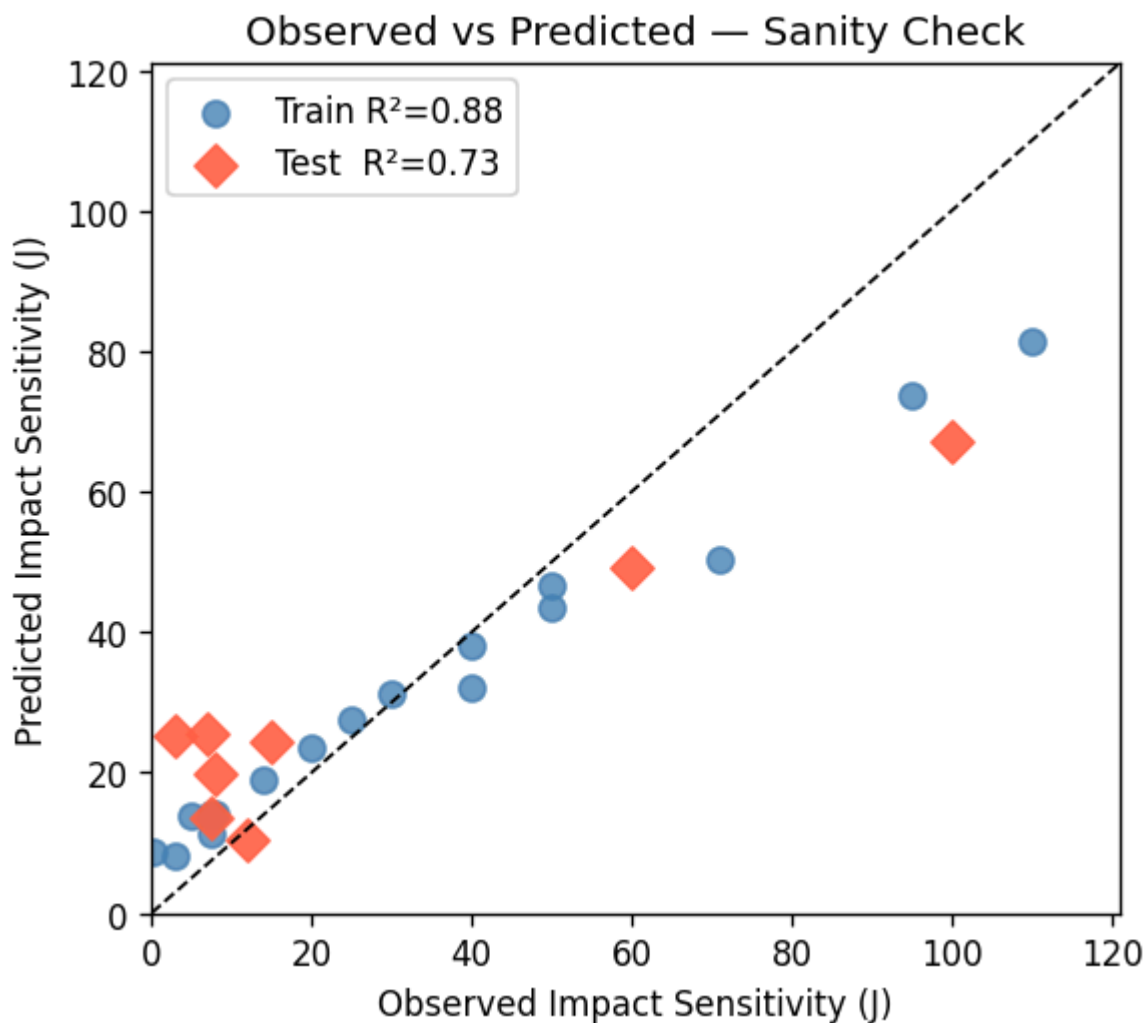
lims = [0, df_desc['ImpactSens_J'].max()*1.1]
ax.plot(lims, lims, 'k--', linewidth=1)
ax.set_xlabel('Observed Impact Sensitivity (J)')
ax.set_ylabel('Predicted Impact Sensitivity (J)')
ax.set_title('Observed vs Predicted — Sanity Check')
ax.legend()
ax.set_xlim(lims); ax.set_ylim(lims)
plt.tight_layout()
plt.show()

```

Sanity check — Random Forest on cleaned dataset:

R^2 training : 0.876
 R^2 test : 0.728
 MAE test : 14.23 J

Training R^2 >> Test R^2 means some overfitting — expected with $n=20$.
 Larger datasets and cross-validation address this (see Chapter 4).



Summary — What You Learned in Chapter 3

Block	Concept	Key Python tool
0	Build a 30-compound literature dataset	<code>pandas.DataFrame</code>
1	Data quality audit — missing values, stats, class balance	<code>pd.isna</code> , <code>describe</code> , <code>matplotlib</code>

Block	Concept	Key Python tool
2	Canonicalise SMILES, detect duplicates via InChIKey	<code>MolToSmiles</code> , <code>InchiToInchiKey</code>
3	Visualise distributions, pairplots, correlations	<code>seaborn.pairplot</code> , <code>heatmap</code>
4	Handle missing values — drop, mean, KNN imputation	<code>SimpleImputer</code> , <code>KNNImputer</code>
5	Detect outliers — boxplot, Z-score, Isolation Forest	<code>scipy.stats.zscore</code> , <code>IsolationForest</code>
6	Calculate descriptors + fingerprints, assemble X matrix	<code>Descriptors</code> , <code>rdFingerprintGenerator</code>
7	Remove low-variance & correlated features; scale X	<code>VarianceThreshold</code> , <code>StandardScaler</code>
8	Random and scaffold-based train/test splits	<code>train_test_split</code> , <code>MurckoScaffold</code>
9	End-to-end pipeline: SMILES → ML-ready arrays	All of the above combined

Next → Chapter 4: Training, tuning, and comparing QSPR models.

References & Tools

- RDKit: <https://www.rdkit.org/docs>
- Scikit-learn preprocessing: <https://scikit-learn.org/stable/modules/preprocessing.html>
- Scikit-learn model selection: https://scikit-learn.org/stable/modules/model_evaluation.html
- Murcko scaffold splitting: <https://www.rdkit.org/docs/source/rdkit.Chem.Scaffolds.MurckoScaffold.html>
- Dobratz, B.M. *LLNL Explosives Reference Database*, 1981
- Cooper, P.W. *Explosives Engineering*, Wiley-VCH, 1996
- Zeman, S. *Sensitivities of High Energy Compounds*, Elsevier, 2003
- Karthikeyan & Vyas *Practical Chemoinformatics*, Springer, 2014

In []:

In []: